CO2 ASSESSMENT TOOL _2

NAME: M.SAI HARSHITHA
REG.NO: 192424408
COURSE CODE: DSA0304s
COURSE NAME: NATURAL LANGUAGE PROCESSOR

CODE:

```
words = ["analyzing", "analysis", "analytical"]
rules = {
   "analyzing": ("analyze", "-ing", "Inflectional"),
   "analysis": ("analysis", "-", "Derivational (lexical mapping)"),
   "analytical": ("analyt", "-ical", "Derivational")
}
print(f"{'Word':<15}{'Root':<15}{'Affix':<12}{'Type':<30}{'Normalized'}")
for word in words:
   root, affix, kind = rules[word]
   print(f"{word:<15}{root:<15}{affix:<12}{kind:<30}analyze")
```

OUTPUT:

```
Word           Root           Affix       Type                           Normalized
analyzing      analyze        -ing        Inflectional                   analyze
analysis       analysis       -           Derivational (lexical mapping)analyze
analytical     analyt         -ical       Derivational                   analyze
```

QUESTION 2:

CODE:

```
words = ["disagree", "agreement", "agreeable"]
rules = {
   "disagree": ("dis-", "agree", "-", "Derivational",
            "Expresses a difference of opinion"),
   "agreement": ("-", "agree", "-ment", "Derivational",
             "State or result of agreeing"),
   "agreeable": ("-", "agree", "-able", "Derivational",
             "Pleasant or acceptable")}
print(f"{'Word':<15}{'Prefix':<10}{'Root':<10}{'Suffix':<10}{'Type':<15}{'Meaning'}")
for word in words:
   prefix, root, suffix, kind, meaning = rules[word]
   print(f"{word:<15}{prefix:<10}{root:<10}{suffix:<10}{kind:<15}{meaning}")
```

OUTPUT:

```
Word          Prefix    Root      Suffix    Type          Meaning
disagree      dis-      agree     -         Derivational  Expresses a difference of opinion
agreement     -         agree     -ment     Derivational  State or result of agreeing
agreeable     -         agree     -able     Derivational  Pleasant or acceptable
```

QUESTION 3:

CODE:

```
words = ["govern", "government", "governance"]
rules = {
   "govern": ("govern", "-", "Level 0: Base"),
   "government": ("govern", "-ment", "Level 1: govern -> government"),
   "governance": ("govern", "-ance", "Level 1: govern -> governance")}
print(f"{'Word':<15}{'Root':<12}{'Affix':<10}{'Hierarchy':<35}{'Normalized'}")
for word in words:
   root, affix, hierarchy = rules[word]
   print(f"{word:<15}{root:<12}{affix:<10}{hierarchy:<35}govern")
```

OUTPUT:

```
Word          Root        Affix     Hierarchy                          Normalized
govern        govern      -         Level 0: Base                      govern
government    govern      -ment     Level 1: govern -> government      govern
governance    govern      -ance     Level 1: govern -> governance      govern
```

QUESTION 4:

CODE:

```
words = ["activate", "activation", "reactivation"]
rules = {
   "activate": ("-", "act", "-ivate", "act -> activate",
            "Verb: to make active"),
   "activation": ("-", "act", "-ivation",
             "act -> activate -> activation",
             "Noun: process of activating"),
   "reactivation": ("re-", "act", "-ivation",
              "act -> activate -> activation -> reactivation",
              "Noun: process of activating again")
}
print(f"{'Word':<15}{'Prefix':<10}{'Root':<10}{'Suffix':<12}{'Normalized':<15}{'Derivational
sequence'}")
```

```
for word in words:
    prefix, root, suffix, sequence, meaning = rules[word]
    print(f"{word:<15}{prefix:<10}{root:<10}{suffix:<12}{'activate':<15}{sequence}")
    print("Meaning:", meaning)
```

OUTPUT:

```
Word           Prefix    Root      Suffix      Normalized     Derivational sequence
activate       -         act       -ivate      activate       act -> activate
Meaning: Verb: to make active
activation     -         act       -ivation    activate       act -> activate -> activation
Meaning: Noun: process of activating
reactivation   re-       act       -ivation    activate       act -> activate -> activation ->
reactivation
Meaning: Noun: process of activating again
```

QUESTION 5:

CODE:

```
words = ["create", "creates", "creating"]
rules = {
    "create": ("-", "Base form", "create"),
    "creates": ("-s", "Third-person singular present", "create"),
    "creating": ("-ing", "Present participle", "create")
}
print(f"{'Word':<15}{'Suffix':<10}{'Grammatical category':<35}{'Root':<12}{'Normalized'}")
for word in words:
    suffix, category, root = rules[word]
    print(f"{word:<15}{suffix:<10}{category:<35}{root:<12}{root}")
```

OUTPUT:

```
Word           Suffix    Grammatical category                Root        Normalized
create         -         Base form                           create      create
creates        -s        Third-person singular present       create      create
creating       -ing      Present participle                  create      create
```